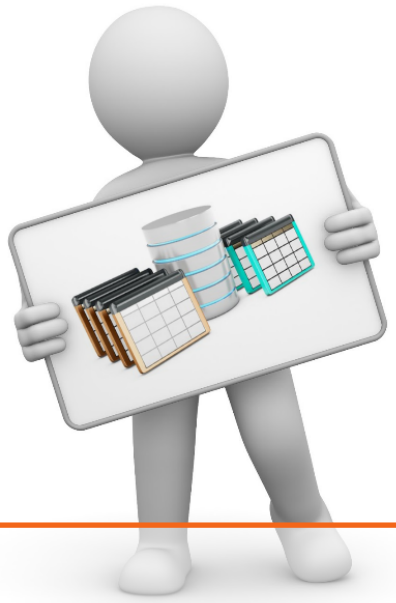


2. Modelo relacional

2.1 Introdução

Um modelo de dados é um conjunto de conceitos e regras que permitem descrever, em diferentes níveis de abstração, a estrutura de uma bases de dados. Poderíamos definir como níveis de abstração, o modelo externo, o modelo global e o modelo interno. O primeiro descreveria a forma como a base de dados seria "vista"por cada um dos seus utilizadores, o segundo, a forma como os dados são descritos para o conjunto dos utilizadores e por fim, o terceiro descreveria a forma como os dados são organizados em termos físicos.

Os modelos globais são extremamente importantes, uma vez que permitem uma maior grau de intervenção na definição de uma base de dados. Os modelos externos, dependentes das linguagens de programação escolhidas, são na sua generalidade semelhantes, e os modelos internos, são ao contrário propriedade de cada uma das soluções particulares implementadas pelos fabricantes.

Os modelos globais podem ser classificados em modelos conceptuais e modelos convencionais.

- **Modelos conceptuais**
  Também denominados por modelos de alto nível, têm como finalidade a descrição global do conjunto de informação, com independência da "máquina". Os seus conceitos são o mais próximos possível do mundo real (entidades, características, etc). São modelos de análise e não de implementação.

- **Modelos convencionais**
  São modelos característicos de cada um dos SGBD[10] e têm como finalidade descrever os dados a nível lógico para cada implementação particular. Tabelas ou relações para o modelo relacional, redes para o modelo em rede e árvores para o modelo hierárquico.

O modelo de dados é a ferramenta utilizada para criar o esquema da base de dados, entendendo-se esquema como a descrição da estrutura da base de dados. O esquema de uma base de dados é distinto da ocorrência de um esquema. Neste caso estamos em presença dos dados armazenados num esquema num dado instante no tempo. O esquema de uma base de dados é invariante no tempo,

[10] Sistema de Gestão de Bases de Dados. É também comum em literatura em língua portugeuesa apresentar o acrónimo da lingua inglesa, DBMS, Database Management System.

pelo menos enquanto a realidade que pretende descrever não varie, enquanto uma ocorrência do esquema, por norma varia ao longo do tempo.

Definição 2.1.1 — Modelo de dados. Conjunto de conceitos, regras e convenções que permitem descrever e manipular os dados de um determinado universo real que pretendemos armazenar numa base de dados.

Num modelo de dados podemos identificar características estáticas e características dinâmicas. As primeiras, como o próprio nome indica, não variam, ou variam muito pouco, com o tempo e estão relacionadas com a estrutura, as segundas, relacionam-se com as operações que se aplicam aos dados.

2.1.1 Características estáticas de um modelo de dados

Cada modelo possui o seu conjunto de características próprias, mas podem identificar-se algumas que são genericamente comuns a todos. Para além desse facto, pode considerar-se a existência de elementos permitidos e elementos proibidos. Elementos permitidos:

- Objetos - entidades, relações, registos, etc;
- Associações entre objetos - relacionamentos, conjuntos, subconjuntos, etc;
- Propriedades dos objetos ou dos relacionamentos - atributos, campos, informação dos dados, etc;
- Domínios - conjunto de valores sobre os quais são definidas as propriedades.

A forma como estes elementos se representam depende de cada um dos modelos concretos. No modelo hierárquico e no modelo de rede, a representação é efetuada por meio de grafos, enquanto que no modelo relacional é efetuada através de tabelas. No modelo E-R[11] usam-se as duas representações.

Os elementos proibidos, que se designam por **restrições**, devem-se ao facto de que no mundo real, nem todos os valores, ou alterações de valores, são possíveis. Por exemplo, a nota de uma disciplina não pode ser superior a 20 ou, num outro exemplo, uma criança de 5 anos, não pode ser pai de um jovem de 20. Para além das restrições impostas pelo universo do qual se pretende armazenar informação, existem outras restrições que reportam a cada modelo de dados concreto. Por exemplo no modelo relacional, não é permitido que existam duas linhas iguais na mesma tabela.

As restrições impostas por cada modelo de dados concreto, dizem-se inerentes e variam de modelo para modelo e por norma são inultrapassáveis . As restrições impostas pelo universo real, são restrições de integridade ou semânticas que podem ser alvo de interpretações do projetista em relação ao problema que tenta resolver.

2.1.2 Características dinâmicas de um modelo de dados

No tempo t_i existirá uma ocorrência do esquema BD_i que não é mais do que o contexto de dados existente no tempo t_i. No momento t_j existirá uma ocorrência BD_j. Caso tenha havido modificações de algum valor na base de dados, então $BD_i \neq BD_j$, sendo que no entanto, ambas as ocorrências BD_i e BD_j devem cumprir com as restrições de integridade ou outras impostas pelo modelo.

A componente dinâmica do modelo resulta do facto de existirem um determinado número de operadores definidos sobre a estrutura do modelo de dados. Todas as estruturas, para um modelo concreto, deverão suportar o mesmo tipo de operações. Uma operação sobre uma ocorrência, transforma esta numa outra ocorrência: $O(BD_i) = BD_j$.

[11] Entidade-Relacionamento.

2.2 Modelo relacional

Em 1969 Edgar Frank Codd, investigador da IBM, escreve um artigo[12] sobre os SGBD que teve como consequência o aparecimento de um novo modelo de dados baseado na teoria das relações. Neste novo modelo, os dados são estruturados em relações(tabelas), sendo que um dos objetivos principais seria manter independência da estrutura face ao método de armazenamento ou outras características do sistema físico.

Em termos de objetivos, Codd propôs o seguinte:

- **Independência física**
  A estrutura de armazenamento dos dados não deve apresentar qualquer influência na sua manipulação lógica, tendo como consequência que os utilizadores não necessitam de alterar os seus programas, mesmo que haja alterações na forma como fisicamente os dados são armazenados.

- **Independência lógica**
  Acrescentar, modificar ou eliminar elementos da base de dados não deve ter influência nos programas ou ações dos utilizadores que estão a aceder a subconjuntos parciais dos dados.

- **Flexibilidade**
  Poder fornecer a cada utilizador os dados da forma mais adequada às suas aplicações.

- **Uniformidade**
  Como a estrutura lógica dos dados é a tabela, estas apresentam forma uniforme, o que facilita o projeto e a manipulação da base dados por parte dos utilizadores.

- **Simplicidade**
  Para além da forma utilizada para a estrutura lógica (tabelas) ser bastante simples, Codd pretendia o desenvolvimento de uma linguagem de interrogação simples, fácil de compreender e de utilizar.

O modelo relacional baseia-se na relação como elemento de base e que visualmente pode ser representado por meio de uma tabela, tal como mostrado na figura 2.2,

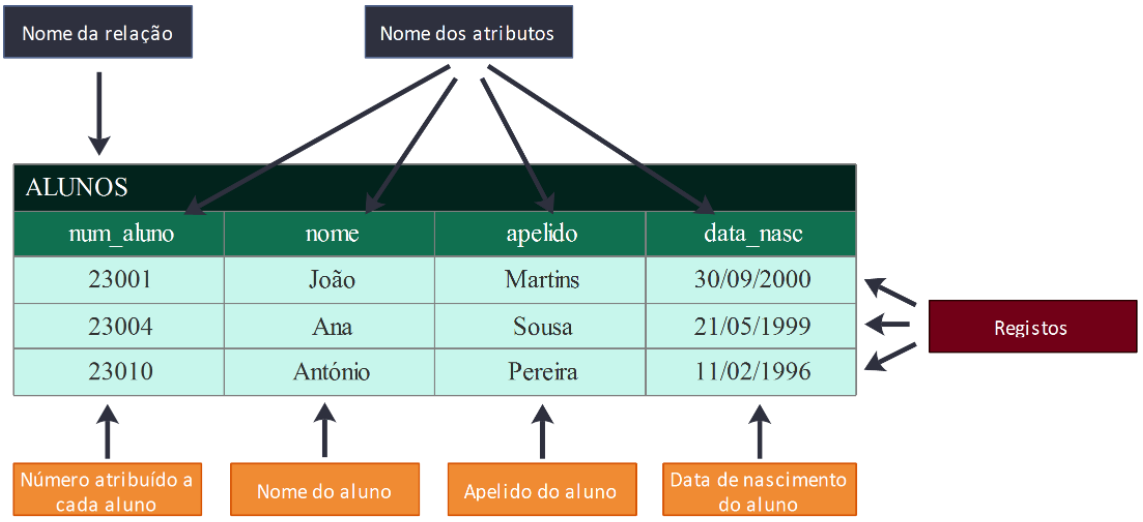


Figura 2.1: Representação de uma relação através de uma tabela

A figura mostra a relação ALUNOS com ênfase nos elementos que compõem o modelo

[12] A relational model of data for large shared data banks.

relacional. O nome da relação, ALUNOS, um conjunto de colunas que representam os atributos, ou características comuns entre os diversos objetos da relação e um conjunto de linhas, também apelidados de tuplos ou registos, que representam os valores para cada uma das características de cada um dos objetos concretos.

Na figura 2.2 para além do exposto podemos ainda observar que na estrutura de uma relação são ainda importantes o domínio dos atributos, conjunto de valores permitidos a cada um dos atributos, o grau de uma relação (número de atributos) e a cardinalidade (número de tuplos).

2.2.1 Domínio

O conceito de domínio é extremamente importante no modelo relacional podendo ser definido por um conjunto de valores homogéneos e atómicos. Considera-se que os valores são homogéneos porque são todos do mesmo tipo e a atomicidade caracteriza-se pelo facto de não poderem ser divisíveis sem que haja perda de informação. Se considerarmos um atributo **disciplina**, podemos considerar que "Matemática"é um valor atómico. Qualquer decomposição da cadeia de caracteres resultaria em valores sem qualquer sentido no universo das disciplinas.

2.2.2 Tabela

A concretização visual de uma relação é uma tabela. No entanto, a relação tem uma série de características que a distinguem de uma tabela.

- Não são permitidas linhas duplicadas;
- As linhas e as colunas não são ordenadas[13];
- A tabela é bidimensional, significando que na intersecção de uma linha com uma coluna só pode existir um valor.

Apesar de uma relação não ser propriamente uma tabela, devido ao facto de assim serem representadas, tornou a diferença entre conceitos razoavelmente fluída. Na figura 2.2.2 pode observar-se a diferença entre a relação, tabela e ficheiro.

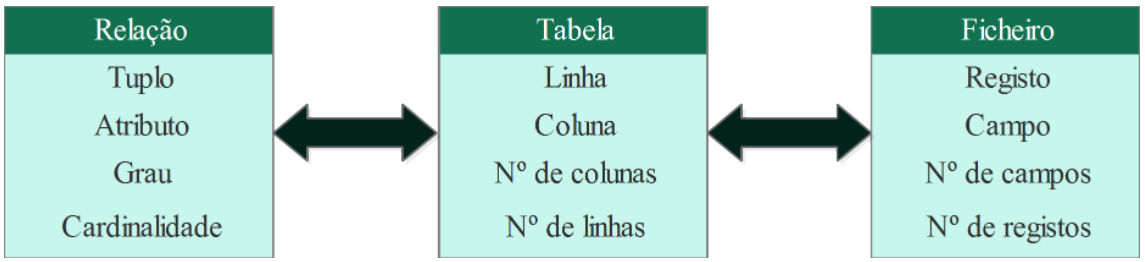


Figura 2.2: Comparação entre relação, tabela e ficheiro

2.2.3 Valores nulos

O conceito de valor nulo, comum a diversos modelos, pode ser definido como marca de ausência de valor, informação inexistente, desconhecida, não aplicável, não válida, não fornecida, indefinida, etc. No contexto das bases de dados a sua utilidade é simples de justificar. As razões para a sua utilização podem ser:

- Adicionar registos para os quais se desconhecem valores para determinados atributos;
- Adicionar um determinado atributo a uma relação já existente. Como é óbvio, todos os registos pré-existentes na tabela terão ausência de valor para o novo registo;

[13] Significa que a ordenação visual obtida é apenas circunstancial e pode ser modificada através de comandos apropriados.

- Atributos que não possuem valor para determinados registos. Exemplo: **carta_de_conducao** para uma criança;

A existência de valores nulos obriga a que tradicionais operações de comparação, operações aritméticas, operações algébricas e operações de agregação sejam definidas de tal forma que possam suportar a utilização de valores nulos. Exemplo: 2>null é ou não verdade? Eventualmente haverá necessidade de introduzir novos operadores que suportem a possibilidade dos atributos conterem valores nulos. Exemplo: **IS NULL** ou **COUNT(IF IS NOT NULL)**.

2.3 Chave candidata e chave primária

Em qualquer base de dados, a informação nela contida deve poder ser identificada através de um qualquer processo. No caso das bases de dados que seguem o modelo relacional, para identificarmos a informação que as tabelas contêm, podem ser usadas as chaves candidatas das relações. No entanto, antes de definir o conceito de chave candidata, deve ser introduzido o conceito de superchave no contexto das relações.

Definição 2.3.1 — Super-chave. Conjunto de um ou mais atributos, que tomados coletivamente permite identificar sem ambiguidades qualquer tuplo de uma relação.

Seja a relação $Alunos(num_aluno, nome, apelido, nif, data_nasc, email, telemovel)$, que dá origem à tabela **alunos** da figura 2.3. Uma super-chave possível para a relação será constituída pelo conjunto de todos os atributos da relação[14]. Ou seja, $\{num_aluno, nome, apelido, nif, data_nasc, email, telemovel\}$ permite identificar cada um dos tuplos da relação. No entanto, existem mais super-chaves para a relação: $\{nif, nome\}$, $\{num_aluno, nome, nif, email, telemovel\}$, $\{nif\}$ ou $\{num_aluno\}$, constituem super-chaves da relação **alunos**.

alunos						
num_aluno	nome	apelido	nif	data_nasc	email	telemóvel
23001	João	Martins	123456789	30/09/2000	joao@nowhere.com	966666666
23004	Ana	Sousa	987654321	21/05/1999	ana@cxcorreio.pt	933333333
23010	António	Pereira	123789456	11/02/1996	antonio@iamhere.edu	911111111

Figura 2.3: Tabela **alunos**

2.3.1 Chave candidata

Do exposto anteriormente é possível concluir que qualquer conjunto que contenha as super-chaves $\{nif\}$ e $\{num_aluno\}$ resulta numa nova super-chave. Tendo o modelo relacional como objetivo a eliminação da redundância, a utilização desnecessária de atributos para identificação dos tuplos é contraditório em relação ao modelo. Daí considerar-se que a chave candidata é o conjunto constituído por um ou mais atributos, que permite identificar de forma unívoca cada tuplo de uma relação, sem que possa ser reduzido sem perder essa qualidade. Apresenta-se a definição de forma sintética.

Definição 2.3.2 — Chave candidata. Uma chave candidata de uma relação é uma super-chave C da relação, de tal forma que não existe nenhum subconjunto de C que seja super-chave.

[14] Por definição, qualquer relação tem pelo menos uma super-chave que é constituída por todos os atributos da relação. Esta caraterística resulta do facto de que no modelo relacional não são permitidos tuplos repetidos.

Do exposto anteriormente resulta que qualquer chave candidata é:

• **Unívoca**
Não podem existir dois tuplos com o mesmo valor para a chave candidata.

• **Irredutível**
Não é possível eliminar atributos da chave candidata sem destruir a sua univocidade.

2.3.2 Chave primária

Numa relação podem existir diversas chaves candidatas, sendo função do projetista escolher uma que funcionará como chave primária. A forma como o projetista deve escolher a chave primária não é definida pelo modelo relacional, sendo no entanto possível apontar alguns critérios que podem ser seguidos por ordem de prioridade:

1. Atributos familiares ao utilizador.
2. Atributos cujo domínio seja numérico.
3. Chaves constituídas por um número menor de atributos.
4. Atributos de preenchimento obrigatório.

Reafirma-se novamente que a escolha de uma chave candidata, na presença de mais do que uma opção, não obedece a qualquer exigência formal, sendo apenas uma decisão a tomar pelo projetista. As chaves candidatas não escolhidas, não têm qualquer papel especial nas tabelas implementadas a partir das relações.

2.4 Integridade de entidade

Os tuplos de uma relação correspondem a instâncias de uma entidade real. Cada instância de uma entidade é identificada de modo único no mundo real. O modelo relacional utiliza a chave primária para identificar cada instância das entidades reais. Se uma chave primária contém um valor nulo[15], significa que não é possível aplicar a definição de chave primária sobre uma determinada entidade, impossibilitando a identificação de uma ou várias das suas instâncias. O facto de poderem existir entidades em que não é possível identificar alguns dos seus elementos, entra em contradição com o próprio conceito de entidade em que cada instância da entidade deve poder ser identificado.

Este raciocínio pode ser estendido às chaves primárias compostas[16] obrigando a que qualquer atributo pertencente à chave primária não possa conter valores nulos. Eis portanto a definição:

Definição 2.4.1 — Integridade de entidade. Se um conjunto de atributos PK[a] é chave primária da relação R, não podem existir tuplos de R com valore nulos para algum dos atributos de PK.

[a] Do inglês, Primary Key

Como consequência, podemos afirmar que numa base de dados relacional não é possível armazenar informação sobre algo que não seja passível de ser identificado.

Existe ainda uma consideração de ordem prática. Como foi referido anteriormente na secção 2.2.3, os valores nulos não devolvem um valor lógico para as operações de comparação de igualdade e desigualdade. "NULL = NULL"não devolve um valor lógico, porque "NULL"não é um valor mas antes um estado. Não se pode afirmar que a comparação anterior seja verdadeira ou falsa. Isto implica que seria impossível estabelecer a univocidade da chave primária, caso fossem permitidos

[15] Valor nulo deve ser entendido como **ausência** de valor. De notar que esta assunção não é entendida da mesma forma em todos os SGBDs.

[16] Terminologia utilizada para chaves primárias constituídas por mais do que um atributo.

valores nulos, uma vez que a verificação dessa propriedade obriga a estabelecer comparações de igualdade.

2.5 Chave forasteira

A maior parte das situações ocorridas no mundo real não se referem apenas a entidades e às suas características ou a acontecimentos e aos seus detalhes. Grande parte da informação no real refere-se à forma como as entidades ou acontecimentos estabelecem ligações ou vínculos entre si.

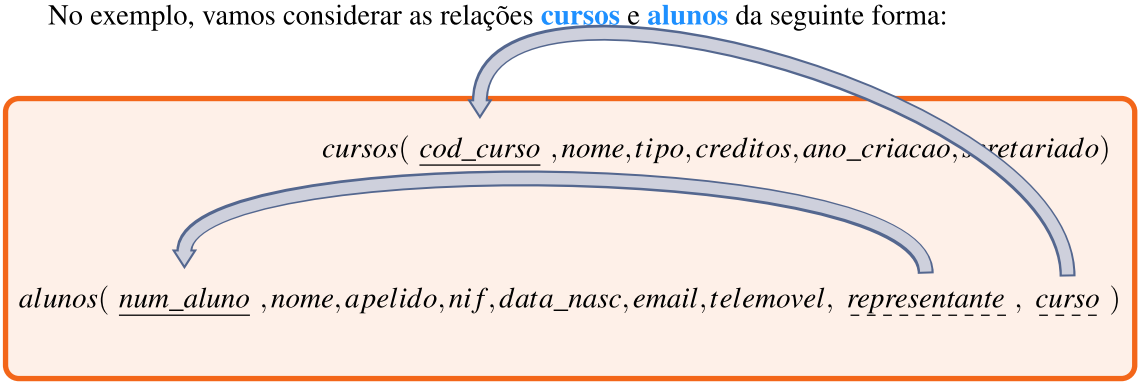
Consideremos a relação **alunos**, anteriormente apresentada na tabela da figura 2.3, que descreve os dados dos alunos de uma determinada instituição de ensino superior. Consideremos ainda a relação **cursos** que pode ser visualizada na tabela da figura 2.4.

cursos					
cod_curso	nome	tipo	creditos	ano_criacao	secretariado
63236	Engenharia Informática	Licenciatura	180	1998	Sandra Marques
59879	Artes Gráficas	Licenciatura	180	1991	Carlos Silva
93005	Automação e Robótica	cTeSP	120	1994	Joana Vidal

Figura 2.4: Tabela **cursos**

No modelo relacional, para além da informação de cada entidade, existe a possibilidade de estabelecer ligações entre os tuplos das relações que corporizam as entidades. No caso das duas relações apresentadas, será eventualmente necessário efetuar ligações da relação **alunos** com tuplos da relação **cursos** de forma a conhecer o curso que cada aluno frequenta. Existem ainda situações em que haverá necessidade de expressar ligações que se estabelecem entre tuplos de uma mesma relação. Por exemplo na relação **alunos** pode ser útil conhecer quais os alunos que são representantes de outros alunos.

No exemplo, vamos considerar as relações **cursos** e **alunos** da seguinte forma:



Na relação **alunos**, para além dos atributos que descrevem características da entidade, está presente um atributo adicional, *curso* que estabelece a ligação entre as duas relações, indicando para cada aluno, o curso a que pertence. Existe ainda um outro atributo adicional,*representante*, que permite efetuar a ligação à chave primária da própria relação e que permite conhecer para cada aluno qual o seu representante nas estruturas da escola.

Nota Por norma as chaves primárias das relações são representadas com o nome dos atributos a sublinhado contínuo e as chaves forasteiras são representadas a sublinhado tracejado.

O mecanismo que permite estabelecer a ligação entre duas relações é denominada de chave forasteira[17] e é constituída por um ou mais atributos que referenciam a chave primária de outra relação. Os valores admissíveis para a chave forasteira ou serão nulos ou devem estar presentes no conjunto de valores utilizados na chave primária referenciada. Qualquer outra situação acarretará ligações incorretas.

Definição 2.5.1 — Chave forasteira. A chave forasteira de uma relação R é um subconjunto de atributos da relação, que se designará por FK[a] e que cumpre as seguintes condições:

1. Existe uma relação S, não necessariamente diferente de R, com chave primária PK.
2. Para todos os tuplos t da relação R, os valores de FK ou são valores nulos ou valores que coincidem com os valores de PK de algum tuplo s de S.

Diz-se então que a chave estrangeira FK referencia a chave primária da relação S e representa-se por:

$$R \xrightarrow{FK} S$$

, sendo FK o conjunto de atributos que constitui a chave primária.

[a] Do inglês, Foreign Key.

Da definição 2.5.1 e do anteriormente exposto acerca do modelo devem ser tidas em conta algumas considerações importantes:

- A definição de chave forasteira não permite concluir que todo o conjunto de valores da chave primária deva estar representado no conjunto de valores da chave forasteira. Basta que haja tuplos da relação S que não sejam referenciados por nenhum tuplo da relação R;
- A chave primária e a chave forasteira que a referencia têm que partilhar o mesmo domínio. Numa implementação prática, significa que a chave primária de uma tabela e a chave estrangeira que a referencia terão que ser do mesmo tipo e partilhar eventuais restrições de tamanho;
- Sendo a chave forasteira um atributo, sem qualquer limitações, de uma relação, significa que a chave forasteira também pode fazer parte da chave primária da relação a que pertence.
-

2.5.1 Cadeias de referenciação

Uma relação referenciada, pode por sua vez estabelecer referências a uma outra relação através de um outro conjunto de atributos. Deste modo podem definir-se ligações entre as diversas relações de uma base de dados.

$$R_n \xrightarrow{FK_n} R_{n-1} \xrightarrow{FK_{n-1}} R_{n-2} \longrightarrow \cdots \longrightarrow R_2 \xrightarrow{FK_2} R_1$$

Em certas situações podem surgir ciclos de referenciação, em que uma relação surge mais do que uma vez na cadeia.

$$R_n \xrightarrow{FK_n} R_{n-1} \xrightarrow{FK_{n-1}} R_{n-2} \longrightarrow \cdots \longrightarrow R_2 \xrightarrow{FK_2} R_1 \xrightarrow{FK_1} R_n$$

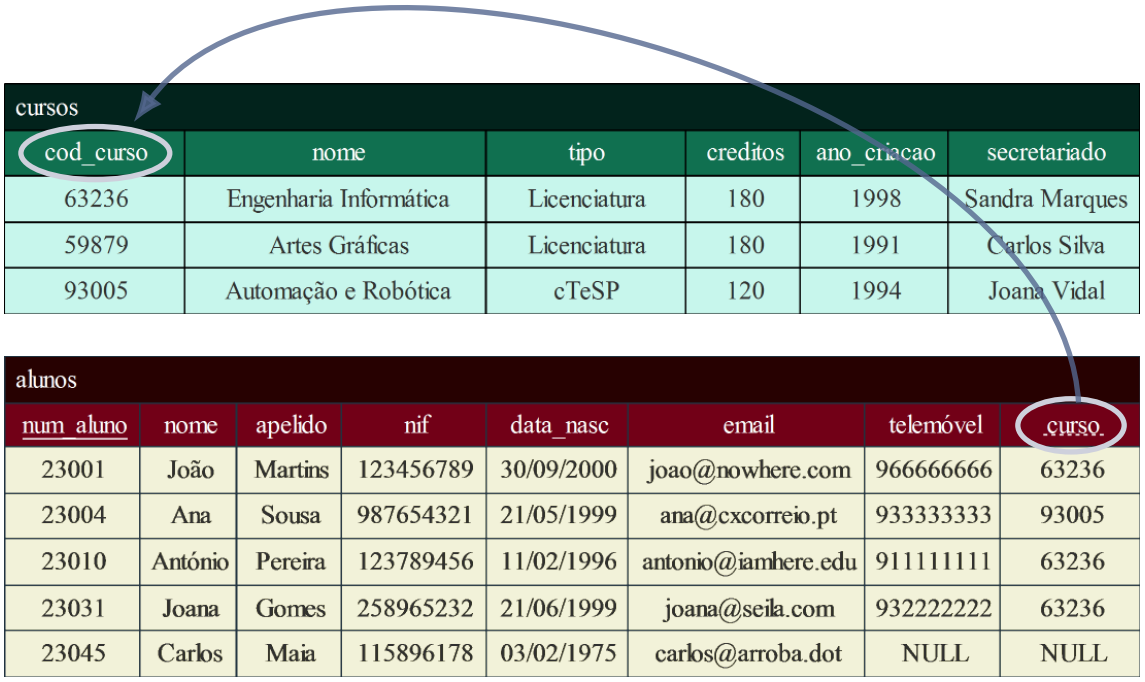
Do caso anterior pode considerar-se o caso particular da relação que se referencia a si mesmo, criando-se uma relação autoreferencial.

$$R_n \xrightarrow{FK_n} R_n$$

[17] Também apelidada de chaves estrangeira ou chave externa

2.6 Integridade referencial

Definição 2.6.1 — Integridade referencial. Se um conjunto de atributos **FK** é uma chave forasteira de uma relação R, que referencia a chave primária **PK** de uma relação S (não necessariamente diferente de R), então para qualquer tuplo t de R, os valores para o conjunto de atributos **FK** de t ou são nulos ou coincidem com os valores de **PK** de algum tuplo s de S.



cursos

cod_curso	nome	tipo	creditos	ano_criacao	secretariado
63236	Engenharia Informática	Licenciatura	180	1998	Sandra Marques
59879	Artes Gráficas	Licenciatura	180	1991	Carlos Silva
93005	Automação e Robótica	cTeSP	120	1994	Joana Vidal

alunos

num_aluno	nome	apelido	nif	data_nasc	email	telemóvel	curso
23001	João	Martins	123456789	30/09/2000	joao@nowhere.com	966666666	63236
23004	Ana	Sousa	987654321	21/05/1999	ana@cxcorreio.pt	933333333	93005
23010	António	Pereira	123789456	11/02/1996	antonio@iamhere.edu	911111111	63236
23031	Joana	Gomes	258965232	21/06/1999	joana@seila.com	932222222	63236
23045	Carlos	Maia	115896178	03/02/1975	carlos@arroba.dot	NULL	NULL

Figura 2.5: Relação alunos e cursos com chave forasteira *alunos* $\xrightarrow{curso}$ *cursos*

No caso da figura 2.5, a relação **alunos** possui uma chave forasteira *alunos* $\xrightarrow{curso}$ *cursos* que referencia a relação **cursos**. Os valores que o atributo *curso* assume, não só são do mesmo domínio, como estão dentro da gama de valores do atributo *cod_curso* da relação **cursos** ou então são nulos. Por exemplo, o aluno 'António Pereira', cujo número é o '23010', tem como valor para o atributo *curso* o valor '63236' o que significa que frequenta o curso de 'Engenharia Informática', como se pode verificar na relação **cursos**.Já por exemplo o aluno 'Carlos Maia' não possui valor para o atributo *curso*, significando eventualmente que é um aluno externo ou uma aluno a frequentar disciplinas integrado no programa "Erasmus". O que garantidamente sabemos é que ele não pertence a curso nenhum. Dita então a regra da integridade referencial, tal como explicitado na definição 2.6.1 que o que não é possível é existir um valor para o atributo *curso*, que não exista previamente para o atributo *cod_curso* de algum tuplo de **cursos**.

Pode então afirmar-se que uma base de dados não pode conter para qualquer das suas chaves forasteiras valores sem concordância, ou seja, para qualquer valor não nulo de uma chave forasteira existe um valor associado na chave primária da relação referenciada.

2.6.1 Regras e limites da chave forasteira

Deve notar-se que a integridade referencial, ou regra de integridade referencial como frequentemente é referida, é definida sobre um determinado estado (instância) da base de dados. Na transição a um novo estado da base de dados, a regra não define como se devem evitar estados incorretos. Fica a cargo do SGBD a forma como se garante que na transição de uma instância a outra seja cumprida a regra. Podem considerar-se duas possibilidades:

- O SGBD recusará qualquer operação que viole a regra;

- O SGBD aceitará as operações, mas acomoda-las-á de forma a garantir a transição para um estado que cumpra a regra.

Algumas questões são importantes para o funcionamento correto do mecanismo da chave forasteira e o seu respeito pela regra da integridade referencial.

- **Valores nulos**
  A chave forasteira pode aceitar valores nulos?
- **Inserções**
  O que sucede quando se insere um tuplo numa relação que possui uma chave forasteira, com um valor não conforme?
- **Eliminaçãoo**
  O que acontece quando se apagam tuplos cujos valores de chave primária são referenciados por valores de chaves forasteiras?
- **Atualização**
  O que sucede quando se modifica um valor da chave primária que é referenciado por um valor de chave forasteira?

No primeiro caso, a possibilidade de aceitar valores nulos depende essencialmente dos requisitos da própria base de dados e do problema que se pretende modelar, mas deve ser levado em conta que a existência de valores nulos numa chave forasteira significam que as cadeias de referenciação podem ser interrompidas para aqueles tuplos onde os valores surjam.

No caso das inserções em relações que possuam chaves forasteiras, deve ser notado que a inserção de valores não conformes para as chaves forasteiras, apenas afeta a relação onde tal facto ocorreu, sem que haja qualquer influência na integridade de outras relações ou de deturpação de ligações já existentes. Por isso é conveniente que um SGBD relacional reaja, recusando a inserção de tuplos que não respeitem a regra da integridade referencial.

Para os casos da atualização de valores na chave primária e eliminação de tuplos em tabelas referenciadas, a pura e simples recusa pode ser de alguma forma "suavizada", uma vez que as operações podem fazer sentido, mas a regra de integridade referencial pode tornar-se demasiado restritiva. Assim um SGBD relacional possibilita três políticas[18] distintas que podem ser aplicadas nos casos já referidos: restrição, atualização em cascata e anulação.

Restrição

Nesta política, o SGBD não aceita operações que eliminem tuplos cujos valores sejam referenciados por chaves forasteiras. Da mesma forma, não aceita que algum dos atributos de uma chave primária de um tuplo seja modificado se existir algum chave forasteira que o referencie.

Atualização em cascata

A política de atualização em cascata permite que as operações de eliminação de registos sejam efetuadas, mesmo que esses registos possuam valores para os atributos da chave primária referenciados pela chave forasteira de uma outra relação. Se um tuplo s de uma relação S, e que é referenciado por um conjunto não nulo de registos de uma relação R que referencia S, for eliminado, então serão eliminados todos os tuplos de R que referenciam t.

Da mesma forma, a atualização em cascata permite que operações de modificação de valores da chave primária sejam efetuadas, mesmo que esses registos possuam valores para os atributos da chave primária referenciados pela chave forasteira de uma outra relação. Se um tuplo s de uma relação S, e que é referenciado por um conjunto não nulo de registos de uma relação R que

[18] Alguns SGBDs acrescentam eventualmente outras possibilidades, mas as três referidas são comuns a todos os SGBDs relacionais.

referencia S, sofrer uma modificação dos valores dos atributos pertencentes à chave primária, então os valores da chave forasteira de todos os tuplos de R que referenciam t serão modificados de acordo.

Anulação

Esta política permite que as operações de atualização e eliminação se efetuem de forma similar ao caso anterior. No entanto, quer para o caso da atualização, quer para o caso da eliminação os valores da chave forasteira são modificado de forma a assumirem o valor nulo (NULL). Não são eliminados nenhuns registos, no caso da eliminação.

É no entanto de notar que esta política só pode ser implementada, caso os atributos que compõem a chave forasteira possam admitir valores nulos.

2.7 Integridade de domínio

Definição 2.7.1 — Integridade de domínio. A regra da integridade de domínio estabelece duas condições:

1. Um valor não nulo de um atributo a_i tem de pertencer ao domínio do atributo a_i.
2. Os operadores que podem aplicar-se sobre um atributo a_i dependem do domínio dos valores de a_i, ou seja, cada operador só pode aplicar-se sobre valores que tenham domínios que lhe sejam adequados.

A primeira condição estabelece que não é possível aceitar valores fora do domínio dos atributos. Por exemplo na relação **alunos** da figura 2.3 não é possível adicionar um registo cujo valor para o atributo *num_aluno* seja "Luís Pais".

A segunda condição estabelece que não é possível utilizar operadores desajustados do domínio dos valores. Usando mais uma vez o exemplo da relação **alunos**, será impossível verificar se $num_aluno >= '\, LusPais'$, uma vez que o operador $>=$ não consegue comparar um atributo cujo domínio é constituído por valores inteiros, com um valor que é uma cadeia de carateres.